AREST: Applicative REpresentational State Transfer

Composing Backus's AST with Fielding's REST
for Specification-Equivalent Application Frameworks

Samuel Lippert
Drivly Inc

Abstract

We present AREST (**A**pplicative **RE**presentational **S**tate **T**ransfer), a compositional architecture that unifies Backus's Applicative State Transition systems (1978) with Fielding's Representational State Transfer (2000). Domain knowledge is expressed as first-order logic propositions in controlled natural language (FORML 2 readings), compiled into executable functions via Backus's combining forms, and rendered as hypermedia representations with state machine transitions as navigation links. The representational state being transferred is the applicative state being transformed. We demonstrate that every transformation in this pipeline is deterministic, lossless, and invertible: the natural language reading is simultaneously the specification, the executable, and the proof obligation.

1 Introduction

Application development produces two artifacts, a specification and an implementation, that diverge over time. The specification states "each customer has exactly one email"; the implementation enforces this through database constraints, API validation, frontend checks, and tests. These are four independent projections of the same proposition that can disagree. Schema design, API frameworks, rule engines, and formal methods each address a fragment of this problem; none spans the full path from domain proposition to running system.

We observe that this divergence arises from treating specification and implementation as separate artifacts. If the sentence

Each Customer has exactly one Email.

is simultaneously a human-readable requirement and a machine-executable uniqueness–mandatory constraint, every transformation in the pipeline is deterministic, lossless, and invertible.

AREST achieves this by composing two state transfer systems that operate at orthogonal layers:

Definition 1 (AST)**.** An *Applicative State Transition* system [1] consists of a set of objects O, a set of functions $F : O \to O$, and a single operation, application: $f : x$. The state $s \in O$ is transformed by applying a command: $s_{n+1} = f : s_n$. Programs are point-free compositions of combining forms. No variables are named; no state is mutated.

Definition 2 (REST)**.** A *Representational State Transfer* system (Fielding, 2000) communicates state between distributed components by transferring representations via a uniform interface. *Hypermedia as the engine of application state* (HATEOAS): each response includes links encoding valid state transitions.

Definition 3 (AREST)**.** An *Applicative REpresentational State Transfer* system is the composition AST∘REST: the representational state being transferred is the applicative state being transformed, rendered as hypermedia with transition links.

2 Theoretical Foundations

AREST synthesizes several theoretical traditions.

2.1 First-Order Predicate Logic

Domain knowledge in AREST is expressed as propositions in first-order logic (FOL):

$$\forall p \, \forall c_1 \, \forall c_2 \, (\text{born}(p, c_1) \wedge \text{born}(p, c_2) \rightarrow c_1 = c_2) \tag{1}$$

Equation (1) is verbalized as: "Each Person was born in at most one Country." The verbalization and the formula are isomorphic, the same proposition in two notations.

2.2 Lambda Calculus, FP Algebra, and AST

Church's lambda calculus [2] provides the computation model: a fully applied term (all roles bound) is a fact, a partially applied term is a query, and evaluation is beta reduction. Backus [1] lifted this to function-level programming: programs built from combining forms without naming variables.

Definition 4 (Combining Forms)**.** Let $f, g, f_1, \ldots, f_n$ be functions and x an object.

$$[f_1, \ldots, f_n] : x = \langle f_1 : x, \ldots, f_n : x \rangle \qquad \text{(Construction)} \tag{2}$$

$$(f \circ g) : x = f : (g : x) \qquad \text{(Composition)} \tag{3}$$

$$(\alpha f) : \langle x_1, \ldots, x_n \rangle = \langle f : x_1, \ldots, f : x_n \rangle \qquad \text{(Apply-to-All)} \tag{4}$$

$$(/f) : \langle x_1, \ldots, x_n \rangle = f : \langle x_1, /f : \langle x_2, \ldots \rangle \rangle \qquad \text{(Insert)} \tag{5}$$

$$(p \rightarrow f; g) : x = \begin{cases} f : x & \text{if } p : x = T \\ g : x & \text{if } p : x = F \end{cases} \qquad \text{(Condition)} \tag{6}$$

$$(\text{bu } f \; a) : x = f : \langle a, x \rangle \qquad \text{(Binary-to-Unary)} \tag{7}$$

In AREST, these forms have precise domain interpretations:

Backus Form	**AREST Interpretation**
Construction $[f_1, \ldots, f_n]$	Graph Schema (CONS of Roles)
Selector s	Role at position s
Composition $f \circ g$	Derivation rule chain
Apply-to-All αf	Population traversal
Insert $/f$	Aggregation (exists, forall)
Condition $p \rightarrow f; g$	Constraint evaluation, guards
Binary-to-Unary bu f x	Partial application (query)
Constant $\bar{x}$	Literal value in a reading

Backus's AST extends FP with explicit state: the system state is an object s, and a program p transforms it: $s' = p : s$. The system evolves by successive application.

Definition 5 (Population)**.** A *population* is a set of ground facts: base facts asserted by commands and derived facts produced by forward chaining. The population is the complete state of the system. Every externally observable value, including entity attributes, state machine statuses, and HATEOAS links, is derivable from the population by pure function application.

2.3 Object-Role Modeling

ORM2 [5] models information as elementary facts, where objects play roles in predicates. The surface syntax, FORML 2 [6], is a controlled natural language:

```
Organization(.Slug) is an entity type.
Organization has Name.
  Each Organization has exactly one Name.
```

Each sentence is a FOL proposition. The reference scheme `(.Slug)` defines the entity's identity predicate. The constraint `Each Organization has exactly one Name` combines a uniqueness constraint (UC, "at most one") with a mandatory constraint (MC, "at least one"). The CSDP (Conceptual Schema Design Procedure) [5] provides a 7-step procedure guaranteeing semantic completeness, and the RMAP (Relational Mapping Procedure) [5] maps conceptual schemas to relational schemas.

The key property: the verbalization is lossless. Every ORM2 constraint has exactly one canonical verbalization pattern, and every well-formed verbalization maps to exactly one constraint. The grammar is restrictive enough to parse deterministically.

ORM2 models state machines as facts about facts. A State Machine instance is a separate entity from the Resource it governs, connected by the fact type `State Machine is for Resource` (1:1).

RMAP maps conceptual schemas to Codd's relations [3]: uniqueness constraints become keys, mandatory constraints become NOT NULL. In AREST, each entity instance is an independent storage unit keyed by its reference scheme, satisfying third normal form.

2.4 REST

Fielding's architectural style [4] constrains distributed systems to transfer representations of state via a uniform interface. In AREST, the state machine's valid transitions from the current status serve as the HATEOAS links. The API response for an entity in status "Draft" includes actions `[place, cancel]`, which are transitions compiled from the readings.

3 The AREST Execution Model

3.1 Commands as Function Application

Every API interaction is a Command: a function from population to population.

$$\text{Command} : \text{Population} \to (\text{Population}', \text{Representation}) \tag{8}$$

A CreateEntity command is a composed function that extends the population with new formulae:

$$\text{create} = \text{emit} \circ \text{derive} \circ \text{validate} \circ \text{resolve} \tag{9}$$

where:

- resolve: applies the reference scheme selector to determine the entity's identity
- validate: evaluates constraints against the current population
- derive: forward-chains derivation rules to fixed point, adding derived formulae (including state machine initialization) to the population
- emit: produces the new population containing the entity, its state machine instance, and all derived facts

State machine initialization is not a separate step. The state machine definition is a set of formulae in the population. When the entity is created, the derivation rules produce the State Machine instance and its initial Status as derived facts. The composition is compiled from the readings. The engine applies it as one reduction.

3.2 State Machines as Fold

A state machine is Backus's Insert (fold) over an event stream:

$$\text{run_machine} = /\text{transition} : \langle e_1, \ldots, e_n \rangle \tag{10}$$

Each transition is a Condition:

$$\text{transition} = (p \to \bar{s'}; \text{id}) \tag{11}$$

where $p = \text{eq} \circ [\text{id}, \overline{\langle s_{\text{from}}, e \rangle}]$ checks whether the current state and event match, and $\bar{s'}$ is the constant function returning the target state.

The transition table compiles to nested conditions. The first matching condition fires; the rest fall through to identity (state unchanged):

$$f = (p_1 \to \bar{s_1};\ (p_2 \to \bar{s_2};\ \ldots;\ \text{id})) \tag{12}$$

Guards are constraints that must produce zero violations for the transition to fire:

$$\text{guarded_transition} = (\text{guard_passes} \wedge p \to \bar{s'};\ \text{id}) \tag{13}$$

where $\text{guard_passes} = \text{null} \circ \text{evaluate_constraints}$.

3.3 Derivation Rules as Policy

Any policy that can be expressed as a derivation rule is evaluated by the engine. Access control, visibility scoping, and domain evolution are all derivation rules compiled to Composition chains. For example, the access rule

```
User accesses Domain iff User has Org Role
  in Organization and Domain belongs to
  that Organization.
```

compiles to:

$$\text{access} = /\text{or} \circ \alpha(\text{bu eq } d_{\text{org}}) : \text{user_orgs} \tag{14}$$

Forward chaining from the user's identity resolves the accessible domains. The derivation rule is the policy.

Domain evolution follows the same pattern. A proposed change to the readings is itself a fact in the population. The change is accepted only if CSDP validation confirms that the resulting schema is consistent.

4 End-to-End Example

We trace a single domain through the full AREST pipeline. An order management domain is declared in FORML 2:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order was placed by Customer.
  Each Order was placed by exactly one Customer.
Order transitions:
  Draft -[Place]-> Placed -[Ship]-> Shipped -[Deliver]-> Delivered
```

The constraint compiles to FOL: $\forall o \forall c_1 \forall c_2\ (placedBy(o, c_1) \wedge placedBy(o, c_2) \rightarrow c_1 = c_2)$. The fact type compiles to a Construction of Selectors: $schema_{placedBy} = [sel_1, sel_2]$. The state machine compiles to nested Conditions (Section 3.2).

A client sends `POST /orders {"customer":"acme"}`. The engine applies $emit \circ derive \circ validate \circ resolve$: *resolve* determines identity from the reference scheme; *validate* checks the UC on *placedBy*; *derive* forward-chains to fixed point, creating a State Machine with initial Status = "Draft"; *emit* returns $\text{Pop}' = \text{Pop} \cup \{Order(\texttt{ord-1}),\ placedBy(\texttt{ord-1}, \texttt{acme}),\ SM(\texttt{ord-1}, \texttt{Draft})\}$. The response:

```
{ "id": "ord-1", "customer": "acme", "status": "Draft",
  "_links": { "place": { "href": "/orders/ord-1/place" } } }
```

The `_links` are projections of the compiled transition table. When the client follows "place," the engine applies $transition : \langle \texttt{Draft}, \texttt{Place} \rangle = \texttt{Placed}$ and the response includes new link `{ship}`, reflecting the updated valid transitions.

If a second command attempts to assign a different Customer to the same Order, the UC on *placedBy* fires and the command is rejected. The response includes the violation as the original reading: "Each Order was placed by exactly one Customer." The population is unchanged.

5 Properties

Theorem 1 (Specification Equivalence)**.** In AREST, the FORML 2 reading is isomorphic to the executable constraint. Every transformation from reading to compiled form is deterministic, lossless, and invertible.

Proof. Each FORML 2 sentence has exactly one canonical verbalization pattern mapping to exactly one FOL formula [6]. The parser produces a schema of propositions. The engine compiles them to combining forms. Evaluation is beta reduction. The transformation is deterministic and invertible: given the compiled form, the original reading can be recovered by verbalization. □

Theorem 2 (Completeness of State Transfer)**.** A CreateEntity command produces the complete new population in one function application: the resource entity, the state machine instance, derived facts, and constraint violations.

Proof. The command is a composition of functions. The composition is applied atomically; no intermediate state is observable. The result contains the complete new population. □

Corollary 1 (HATEOAS from Readings)**.** The hypermedia links in the REST representation are projections of the compiled state machine's transition table. No URL patterns are hardcoded.

Theorem 3 (Derivability)**.** Every externally observable value is derivable from the population by pure function application. No observable state exists outside the population.

The execution model does not require global evaluation. RMAP partitions the population per entity, and commands operate on a bounded subpopulation determined by role reachability. All combinators, including Insert, execute over finite, entity-scoped sets. Locality is a consequence of the schema, not of the runtime.

6 Related Work

Datalog [7] shares AREST's bottom-up, fixed-point evaluation but lacks a controlled natural language surface, ORM2's constraint taxonomy, and the REST rendering layer. **Alloy** [8] and **TLA+** [9] verify specifications but do not produce running systems. **CQRS and Event Sourcing** share the command/query separation and "state as fold over events" structure; AREST's difference is that the fold is a compiled combinator, not a hand-written projection. **GraphQL**, **OData**, and **RDF** each address fragments of the problem without closing the specification–implementation gap.

7 Conclusion

AREST composes Applicative State Transition systems with Representational State Transfer. The readings are axioms. The engine applies functions to populations. The API renders the result as hypermedia. The client navigates by following links. Each link is a function application waiting to happen.

The theoretical foundations: first-order logic, Church's computation [2], Codd's relations [3], Backus's algebra [1], Halpin's modeling [5], Fielding's distribution [4]. AREST is their synthesis.

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation of the reference system and as a drafting aid during preparation of this manuscript. All architectural decisions, theoretical contributions, and the composition of AST with REST were made by the human author.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, 1978.

[2] A. Church, "An Unsolvable Problem of Elementary Number Theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936.

[3] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.

[5] T. Halpin, *Information Modeling and Relational Databases*, 2nd ed. Morgan Kaufmann, 2008.

[6] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[7] S. Ceri, G. Gottlob, and L. Tanca, "What You Always Wanted to Know About Datalog (And Never Dared to Ask)," *IEEE Transactions on Knowledge and Data Engineering*, vol. 1, no. 1, pp. 146–166, 1989.

[8] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, revised ed. MIT Press, 2012.

[9] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.